

برنامه‌نویسی به زبان C++ (فصل سوم)

مدرس: مریم خراشادیزاده



فهرست مطالب

- فصل اول : مقدمات زبان C++
- فصل دوم : ساختارهای تصمیم گیری و تکرار
- فصل سوم: آرایه ها و رشته ها
- فصل چهارم: توابع
- فصل پنجم: ساختارها و اشاره گرها





آرایه یک فضای پیوسته از حافظه اصلی کامپیوتر می‌باشد که می‌تواند بیش از یک مقدار را در خود جای دهد. آرایه‌ها در C++ دارای ویژگیهای زیر هستند:

کلیه عناصر یک آرایه از یک نوع داده می‌باشند.

عناصر آرایه به وسیله اندیس (شماره، شاخص) مشخص می‌شوند.

در C++ اندیس آرایه از صفر شروع می‌شود.

Arrays



data-type ArrayName[size]

`int x[5];` اعلان آرایه یک بعدی با عناصری از نوع اعداد صحیح

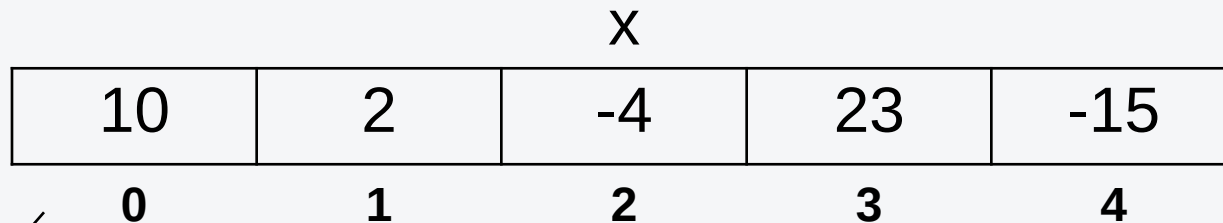
`x[0] = 10;` مقداردهی عناصر آرایه یک بعدی

`x[1] = 2;`

`x[2] = -4;`

`x[3] = 23;`

`x[4] = -15;`



اولین عنصر `x[0]`

پنجمین عنصر `x[4]`

مقداردهی اولیه آرایه‌های یک بعدی

One-Dimensional Array Initialization



```
int x[5] = {4, 2, 5, 17, 30};
```

X

4	2	5	17	30
0	1	2	3	4

```
int x[5] = {12, 5};
```

X

12	5	0	0	0
0	1	2	3	4

```
float x[ ] = {2.4, 6.3, -17.1, 14.2, 5.9, 16.5};
```

X

2.4	6.3	-17.1	14.2	5.9	16.5
0	1	2	3	4	5

مثال



مثال ۱: برنامه زیر ۵ عدد را از ورودی دریافت می‌کند و در آرایه‌ای قرار می‌دهد و سپس این اعداد را چاپ می‌کند.

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    int n[ 5 ]; // n is an array of 5 integers

    // input each array element's value
    for ( int i = 0; i < 5; i++ )
    {
        cout << "Enter array element\'s value [" << i << "]: ";
        cin >> n[ i ]; // input element at location i
    }

    cout << "Element" << setw( 13 ) << "Value" << endl;

    // output each array element's value
    for ( int j = 0; j < 5; j++ )
        cout << setw( 7 ) << j << setw( 13 ) << n[ j ] << endl;

    return 0; // indicates successful termination
} // end main
```

```
Enter array element's value [0]: 3
Enter array element's value [1]: 76
Enter array element's value [2]: 1234
Enter array element's value [3]: 347
Enter array element's value [4]: 40
```

Element	Value
0	3
1	76
2	1234
3	347
4	40

تمرین



1. برنامه‌ای بنویسید که ۱۰۰ عدد را از ورودی دریافت کند و بعد اعداد را با ترتیب معکوس چاپ کند.
2. برنامه‌ای بنویسید که ۲۰ عدد را از ورودی دریافت کند و در آرایه‌ای ذخیره کند و سپس اندیس بزرگترین و کوچکترین عنصر آرایه را چاپ کند.
3. برنامه‌ای بنویسید که آرایه و عددی را از ورودی دریافت کند و بررسی کند که آیا این عدد در آرایه وجود دارد یا نه. در صورت وجود اندیس آن و در صورت عدم وجود ۱- را چاپ کند.
4. برنامه‌ای بنویسید که دو آرایه ۱۰ عنصری را از ورودی دریافت کند و سپس دو آرایه را به دنبال هم در آرایه سومی کپی کند.
5. برنامه‌ای بنویسید که آرایه و عددی را از ورودی دریافت کند و در صورت وجود، آن را از آرایه حذف کند.
6. برنامه‌ای بنویسید که عددی را به آرایه مرتب اضافه کند به شکلی ترتیب اعداد به هم نخورد.
7. برنامه‌ای بنویسید که دو آرایه ۱۰ عنصری را از ورودی دریافت کند و سپس عناصر مشترک دو آرایه را چاپ کند.
8. برنامه‌ای بنویسید که ۲۰ عدد را از ورودی دریافت کند و در آرایه‌ای ذخیره کند و سپس اعداد را مرتب کند.

مثال



مثال ۲ (Linear Search): برنامه زیر ۱۰۰ عدد را از کاربر دریافت می‌کند و اعداد را در آرایه‌ای ذخیره می‌کند، سپس عددی را دریافت می‌کند و جود عدد در آرایه بررسی می‌کند و پیغام مناسب را چاپ می‌کند.

```
#include <iostream>
using namespace std;

int main()
{
    const int size = 100;
    int arr[size];
    int key, index;

    //take array elements
    for ( int i = 0; i < size; i++ )
    {
        cout << "Enter array element\'s value [" << i << "]: ";
        cin >> arr[i] ;
    }

    // take key
    cout << "Enter Search Element: ";
    cin >> key;
```

```
// search key
index = -1;
for (int i = 0; i < size; i++)
{
    if (key == arr[i])
    {
        index = i;
        break;
    }
}
if (index == -1)
    cout << key << " Not Found" << endl;
else
    cout << key << " Found at Index = " << index << endl;

return 0;
}
```


مثال



مثال ۳: برنامه زیر نام و نمرات درس مبانی کامپیوتر یک کلاس را دریافت می‌کند و سپس نام دانشجویان یا دانشجویانی که بیشترین نمره را گرفته‌اند و همچنین نام تمامی دانشجویانی که نمره آنها زیر میانگین کلاس بوده‌اند را نمایش می‌دهد.

```
#include <iostream>
#include <string>

using namespace std;

int main()
{
    int n;
    cout << "Enter the number of students; ";
    cin >> n;

    string Name[n];
    float Score[n];

    for ( int i = 0; i < n; i++ )
    {
        cout << "Enter the student's name and score: ";
        cin >> Name[i] >> Score [i];
    }

    float topScore = Score[0];
    float sum = 0;
```

```
    for (int i = 0; i < n; i++)
    {
        sum += Score[i];
        if (Score[i] > topScore)
            topScore = Score[i];
    }
    float average = sum / n;
    cout << "Students with highest score:" << endl;
    for (int i = 0; i < n; i++)
        if (Score[i] == topScore)
            cout << Name[i] << endl;

    cout << "average = " << average << "\nStudents below
        average:" << endl;

    for (int i = 0; i < n; i++)
        if (Score[i] < average)
            cout << Name[i] << endl;

    return 0;
}
```

آرایه‌های دو بعدی

Two-Dimensional Arrays



- ❖ آرایه‌های دوبعدی هم مانند آرایه‌های یک بعدی، مجموعه‌ای از داده‌ها (از یک نوع داده) را نگهداری می‌کنند با این تفاوت که عناصر آن با دو اندیس مشخص می‌شوند.
- ❖ به عبارت دیگر آرایه‌های یک بعدی را می‌توان مشابه یک لیست و آرایه‌های دو بعدی را می‌توان مانند یک ماتریس در نظر گرفت.
- ❖ می‌توان آرایه‌هایی با ابعاد بالاتر نیز تعریف کرد.

`int a[3][4];`

اعلان یک آرایه دوبعدی با عناصری از نوع اعداد صحیح

	ستون 0	ستون 1	ستون 2	ستون 3
سطر 0	<code>a[0][0]</code>	<code>a[0][1]</code>	<code>a[0][2]</code>	<code>a[0][3]</code>
سطر 1	<code>a[1][0]</code>	<code>a[1][1]</code>	<code>a[1][2]</code>	<code>a[1][3]</code>
سطر 2	<code>a[2][0]</code>	<code>a[2][1]</code>	<code>a[2][2]</code>	<code>a[2][3]</code>

مقداردهی اولیه آرایه‌های دو بعدی

Two-Dimensional Array Initialization



```
int a[3][4]={ {1,2,3,4}, {5,6,7,8}, {9,10,11,12} } ;
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12

```
int  a[3][4]= { {1}, {2,3} , {4,5,6} } ;
```

	0	1	2	3
0	1	0	0	0
1	2	3	0	0
2	4	5	6	0

مقداردهی اولیه آرایه‌های دو بعدی

Two-Dimensional Array Initialization



- ❖ یک آرایه دو بعدی، در حقیقت آرایه‌ای یک‌بعدی است، که داده‌ها به ترتیب به صورت سطر به سطر (یا ستون به ستون) پشت سر هم نگهداری می‌شوند. پس می‌توان آن را به صورت یک‌بعدی هم مقداردهی اولیه کرد.
- ❖ در اعلان آرایه‌های دوبعدی ذکر تعداد ستون‌ها الزامی است.

```
int a[3][4] = {1,2,3,4,5} ;
```

	0	1	2	3
0	1	2	3	4
1	5	0	0	0
2	0	0	0	0

```
int a[ ][4] = {1,7,8,9,3,2,6,10} ;
```

	0	1	2	3
0	1	7	8	9
1	3	2	6	10



دریافت مقادیر عناصر آرایه دو بعدی از ورودی :

```
int x[5][3];
for (int i = 0; i < 5; ++i)
    for (int j = 0; j < 3; ++j)
        cin >> x[ i ][ j ] ;
```

نمایش مقادیر عناصر آرایه دو بعدی :

```
for (int i = 0; i < 5; ++i)
{
    for (int j = 0; j < 3; ++j)
        cout << x[ i ][ j ] << '\t';
    cout << endl;
}
```

تمرین



1. برنامه‌ای بنویسید که یک ماتریس 20×10 را از ورودی دریافت کند و مجموع عناصر آن را چاپ کند.
2. برنامه‌ای بنویسید که یک ماتریس 20×20 را از ورودی دریافت کند و سپس عناصر قطر ماتریس را چاپ کند.
3. برنامه‌ای بنویسید که یک ماتریس 20×20 را از ورودی دریافت کند و سپس بزرگترین عنصر هر سطر را به ترتیب در یک آرایه یک بعدی ذخیره کند.
4. برنامه‌ای بنویسید که در یک ماتریس 10×10 جدول ضرب را ذخیره کند.
5. برنامه‌ای بنویسید که دو ماتریس 20×10 را از ورودی دریافت کند و سپس حاصل جمع دو ماتریس را در ماتریس سومی ذخیره و چاپ کند.
6. برنامه‌ای بنویسید که دو ماتریس 20×20 را از ورودی دریافت کند و سپس حاصل ضرب دو ماتریس را در ماتریس سومی ذخیره و چاپ کند.

مثال



مثال ۴: برنامه زیر حاصلضرب دو ماتریس را محاسبه و چاپ می کند.

```
#include <iostream>
#include <string>

using namespace std;

int main()
{
    int n;
    cin >> n;
    int A[n][n], B[n][n], C[n][n];

    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
            cin >> A[i][j];

    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
            cin >> B[i][j];
```

```
    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
        {
            C[i][j] = 0;
            for (int k = 0; k < n; k++)
                C[i][j] = C[i][j] + A[i][k]*B[k][j];
        }

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
            cout << C[i][j] << '\t';
        cout << endl;
    }

    return 0;
}
```




❖ آرایه‌هایی که تا کنون معرفی شد، توسط زبان C++ به صورت پیش‌فرض و بدون نیاز به کتابخانه‌های C++ تعریف شده‌اند. این آرایه‌ها به عنوان نوع داده‌ای ابتدایی در C++ شناخته می‌شوند و به شما اجازه می‌دهند مجموعه‌ای از عناصر مشابه (مثل اعداد، کاراکترها، و غیره) را ذخیره کنید.

❖ این آرایه‌ها محدودیت‌هایی دارند:

- اندازه آنها ثابت است و قابلیت تغییر اندازه ندارند.
- کنترل و مدیریت داده در آرایه‌های **built-in** به عهده برنامه‌نویس است و قابلیت‌های خودکار برای عملیات مختلف وجود ندارد.
- به عنوان مثال چون کامپایلر کنترلی روی اندیس‌های آرایه ندارد، اندیس‌ها به آسانی می‌تواند از اندازه آرایه فراتر رود.
- نگهداری تعداد داده‌های موجود در آرایه، افزودن و حذف کردن داده‌ها در این آرایه‌ها نیازمند عملیاتی است که باید توسط برنامه‌نویس پیاده‌سازی شود.
- استفاده از عملگر انتساب و همین‌طور عملگرهای مقایسه‌ای هم در آرایه‌ها امکانپذیر نیست.

❖ به همین علت C++ نوع داده‌هایی به نام **array** و **vector** در اختیار برنامه‌نویسان قرار می‌دهد که امکان نگهداری عناصر مشابه با قابلیت‌های بیشتر فراهم می‌کند.

❖ برای استفاده از این نوع داده‌ها باید کتابخانه‌های مربوطه به برنامه ضمیمه شود.



❖ کلاس **array** در **C++** یک ساختار داده‌ای از **C++ Standard Library (STL)** است که برای ذخیره‌سازی مجموعه‌ای از عناصر مشابه با اندازه ثابت استفاده می‌شود. این کلاس از **C++11** به بعد معرفی شده و قابلیت‌های بیشتری نسبت به آرایه‌های داخلی (**built-in arrays**) ارائه می‌دهد.

❖ برای استفاده از این ساختار داده حتما باید کتابخانه **array** به برنامه ضمیمه شود.

```
#include <array>
```

❖ برای ایجاد یک متغیر از نوع این ساختار داده باید به شکل زیر عمل کرد:

```
std::array<type, size> variable_name;
```

❖ **type** نوع داده‌ای که در آرایه ذخیره می‌شود را تعیین می‌کند و **size** تعداد عناصر ساختار را تعیین می‌کند. برای نام متغیر هر نامی که از قواعد نامگذاری شناسه‌ها در **C++** تبعیت کند، می‌توان انتخاب کرد. به عنوان مثال:

```
std::array<int, 5> myArray = {10, 20, 30, 40, 50};
```

❖ در مثال فوق متغیر **myArray** مقداردهی اولیه نیز شده است.

عملگر =

با استفاده از عملگر انتساب می توان آرایه را به آرایه دیگری نسبت داد.

```
array<int, 5> Array1 = {10, 20, 30, 40, 50};  
array<int, 5> Array2 = {1, 2, 3, 4, 5};  
Array1 = Array2;
```

تابع size()

این تابع اندازه آرایه ایجاد شده را برمی گرداند.

```
array<int, 5> myArray = {10, 20, 30, 40, 50};  
cout << myArray.size();
```

5



دسترسی به عناصر با عملگر []

مانند آرایه‌های داخلی با عملگر [] می‌توان به اعضا با استفاده از اندیس دسترسی پیدا کرد. اندیسها هم مشابه آرایه‌های داخلی از صفر شروع می‌شود. این عملگر کنترلی روی بازه اندیسهای مجاز ندارد.

```
array<int, 5> myArray = {10, 20, 30, 40, 50};  
myArray[2] = 100;  
cout << myArray[1] << ' ' << myArray[2];
```

20 100

تابع at(index)

این تابع مشابه عملگر [] عمل می‌کند با این تفاوت که در صورت نامعتبر بودن اندیس یک exception رخ می‌دهد.

```
array<int, 5> myArray = {10, 20, 30, 40, 50};  
myArray.at(2) = 100;  
cout << myArray.at(1) << ' ' << myArray.at(2);
```

20 100



تابع front()

این تابع اولین عنصر آرایه را برمی‌گرداند.

```
array<int, 5> myArray = {10, 20, 30, 40, 50};  
cout << myArray.front();
```

10

تابع back()

این تابع آخرین عنصر آرایه را برمی‌گرداند.

```
array<int, 5> myArray = {10, 20, 30, 40, 50};  
cout << myArray.back();
```

50



تابع fill(value)

این تابع تمامی عناصر آرایه را با مقدار ورودی تابع (value) پر می کند

```
array<int, 5> myArray = {10, 20, 30, 40, 50};  
myArray.fill(100);
```

تابع swap(anotherArray)

این تابع محتوای دو آرایه را تعویض می کند.

```
array<int, 5> Array1 = {10, 20, 30, 40, 50};  
array<int, 5> Array2 = {1, 2, 3, 4, 5};  
Array1.swap(Array2); //Array2.swap(Array1);
```

مثال



مثال ۵: برنامه رو برو نحوه استفاده از کلاس array و قابلیت‌های این کلاس را نشان می‌دهد.

```
int main() {  
    // Define and initialize 5-element.  
    array<int, 5> myArray = {10, 20, 30, 40, 50};  
  
    //Print the array size.  
    cout << "Size of array: " << myArray.size() << endl;  
  
    // Print all array elements.  
    cout << "Elements in array:" << endl;  
    for (size_t i = 0; i < myArray.size(); ++i) {  
        cout << "Element at index " << i << ": " << myArray[i] << endl;  
    }  
  
    // Print the first and last elements.  
    cout << "First element: " << myArray.front() << endl;  
    cout << "Last element: " << myArray.back() << endl;  
  
    // Copy one array to another array.  
    array<int, 5> anotherArray;  
    anotherArray = myArray;  
    cout << "Elements in anotherArray:" << endl;  
    for (int elem : anotherArray) {  
        cout << elem << " ";  
    }  
}
```



```
cout << endl;
```

```
// Set all array elements to a new value (100).
```

```
myArray.fill(100);
```

```
cout << "myArray after fill with 100:" << endl;
```

```
for (int elem : myArray) {
```

```
    cout << elem << " ";
```

```
}
```

```
cout << endl;
```

```
// Swap the values of the array with another array.
```

```
myArray.swap(anotherArray);
```

```
cout << "myArray after swap:" << endl;
```

```
for (int elem : myArray) {
```

```
    cout << elem << " ";
```

```
}
```

```
cout << endl;
```

```
return 0;
```

```
}
```


مثال

خروجی برنامه:

```
"C:\Users\Maryam\courses\Computer Basics\array_library\b...
Size of array: 5
Elements in myArray:
Element at index 0: 10
Element at index 1: 20
Element at index 2: 30
Element at index 3: 40
Element at index 4: 50
First element: 10
Last element: 50
Elements in anotherArray:
10 20 30 40 50
myArray after fill with 100:
100 100 100 100 100
myArray after swap:
10 20 30 40 50

Process returned 0 (0x0)   execution time : 0.203 s
Press any key to continue.
_
```



❖ کلاس **vector** در **C++** یک ساختار داده‌ای از **C++ Standard Library (STL)** است که برای ذخیره‌سازی مجموعه‌ای از عناصر مشابه با اندازه متغیر استفاده می‌شود. این کلاس قابلیت‌های بیشتری نسبت به آرایه‌های داخلی (**built-in arrays**) ارائه می‌دهد.

❖ برای استفاده از این ساختار داده حتما باید کتابخانه **vector** به برنامه ضمیمه شود.

```
#include <vector>
```

❖ برای ایجاد یک متغیر از نوع این ساختار داده باید به شکل زیر عمل کرد:

```
std::vector<type> variable_name;
```

❖ **type** نوع داده‌ای که در آرایه ذخیره می‌شود را تعیین می‌کند و برای نام متغیر هر نامی که از قواعد نامگذاری شناسه‌ها در **C++** تبعیت کند، می‌توان انتخاب کرد. به عنوان مثال:

<code>std::vector<int> vec;</code>	ایجاد یک بردار خالی
<code>std::vector<int> vec2(5);</code>	ایجاد برداری با ۵ عضو (مقادیر اولیه برای اعضا صفر است)
<code>std::vector<int> vec3(5,10);</code>	ایجاد برداری با ۵ عضو (مقادیر اولیه برای اعضا ۱۰ است)
<code>std::vector<int> vec4 = {1, 2, 3, 4, 5};</code>	ایجاد برداری با اعضای نشان داده شده



تابع size()

این تابع تعداد عناصر موجود در بردار را برمی‌گرداند.

```
vector<int> myVector = {10, 20, 30, 40, 50};  
cout << myVector.size();
```

5

تابع capacity()

این تابع ظرفیت بردار (تعداد عناصری که بدون نیاز به افزایش ظرفیت می‌توان در بردار ذخیره کرد) را برمی‌گرداند. توجه کنید این مقدار الزاما برابر تعداد عناصر موجود در بردار نیست.

```
Vector<int> myVector = {10, 20, 30, 40, 50};  
cout << myVector.capacity();
```



تابع `resize(newSize)`

این تابع عناصر موجود در بردار را تغییر می‌دهد. در صورت کاهش اندازه، عناصر بردار از انتهای آن حذف می‌شود و در صورت افزایش اندازه، عناصر (با مقدار پیش‌فرض و یا مقدار مشخص شده) اضافه می‌شوند.

```
vector<int> myVector = {10, 20, 30, 40, 50};  
myVector.resize(10); //myVector.resize(10,100)
```

تابع `reserve(Capacity)`

این تابع ظرفیت بردار (تعداد عناصری که بدون نیاز به افزایش ظرفیت می‌توان در بردار ذخیره کرد) را با مقدار داده شده تنظیم می‌کند. از آنجایی که بردار اندازه ثابتی ندارد با افزایش عناصر ممکن است نیاز به تخصیص مجدد حافظه انتقال داده‌ها به فضای جدید باشد. پس در صورت پیش‌بینی ظرفیت موردنیاز و رزرو آن می‌توان حافظه را بهتر مدیریت کرد و تغییرات مکرر جلوگیری کرد.

```
Vector<int> myVector = {10, 20, 30, 40, 50};  
myVector.reserve(50);
```



دسترسی به عناصر با عملگر []

مانند آرایه‌های داخلی با عملگر [] می‌توان به اعضا با استفاده از اندیس دسترسی پیدا کرد. اندیس‌ها هم مشابه آرایه‌ها از صفر شروع می‌شود. این عملگر کنترلی روی بازه اندیس‌های مجاز ندارد.

```
vector<int> myVector = {10, 20, 30, 40, 50};  
myVector[2] = 100;  
cout << myArray[1] << ' ' << myArray[2];
```

20 100

تابع at(index)

این تابع مشابه عملگر [] عمل می‌کند با این تفاوت که در صورت نامعتبر بودن اندیس یک **exception** رخ می‌دهد.

```
vector<int> myvector = {10, 20, 30, 40, 50};  
myArray.at(2) = 100;  
cout << myArray.at(1) << ' ' << myArray.at(2);
```

20 100



تابع front()

این تابع اولین عنصر بردار را برمی‌گرداند.

```
vector<int> myVector = {10, 20, 30, 40, 50};  
cout << myVector.front();
```

10

تابع back()

این تابع آخرین عنصر بردار را برمی‌گرداند.

```
vector<int> myVector = {10, 20, 30, 40, 50};  
cout << myVector.back();
```

50



تابع begin() و end()

تابع `begin` نشانگری (iterator) به اولین عنصر بردار و تابع `end` نشانگری (iterator) به عنصر بعد از آخرین عنصر بردار را برمی‌گرداند. قطعه کد زیر اعضای بردار را به کمک نشانگرها پیمایش و چاپ می‌کند. `iterator` نوع داده تعریف شده برای ارجاع به اعضای ساختار داده‌هایی مثل `vector`، `array` و `list` هستند، که شباهتهایی به اشاره‌گرها در زبان C++ دارند.

```
vector<int> vec = {1, 2, 3, 4, 5};  
vector<int>::iterator it;  
for (it = vec.begin(); it != vec.end(); ++it)  
    cout << *it << " ";
```

1 2 3 4 5



تابع (push_back(value))

این تابع مقدار داده شده را به انتهای بردار اضافه می‌کند.

```
vector<int> myVector = {10, 20, 30, 40, 50};  
myVector.push_back(100);  
for (int num : myVector)  
    cout << num << " ";
```

10 20 30 40 50 100

تابع (pop_back())

این تابع آخرین عنصر بردار را حذف می‌کند. توجه کنید از آنجایی که حذف آخرین عنصر از بردار خالی رفتار نامشخصی را به دنبال دارد، بهتر است قبل از استفاده از این تابع از خالی نبودن بردار مطمئن باشید.

```
vector<int> myVector = {10, 20, 30, 40, 50};  
myVector.pop_back();  
for (int num : myVector)  
    cout << num << " ";
```

10 20 30 40



تابع insert(position,value)

این تابع مقدار داده شده را به مکان مشخص شده اضافه می‌کند. می‌توان با ارسال دو نشانگر شروع و پایان از یک ساختار داده دیگر مثل آرایه یا بردار به جای مقدار، از تابع **insert** برای افزودن بیش از یک عنصر استفاده کرد.

```
vector<int> myVector = {10, 20, 30};  
myVector.insert(myVector.begin() + 1, 50);  
myVector.insert(myVector.end(), 40);  
for (int num : myVector)  
    cout << num << " ";
```

10 50 20 30 40



تابع (erase(position))

این تابع برای حذف یک یا چند عنصر از بردار استفاده می‌شود. در صورتی که تنها یک مکان مشخص شود تنها داده موجود در موقعیت مشخص شده و در صورت مشخص کردن بیش از یک موقعیت عناصر مابین دو موقعیت حذف می‌شود.

```
vector<int> myVector = {1, 2, 3, 4, 5, 6, 7};  
myVector.erase(myVector.begin()+1);  
myVector.erase(myVector.begin()+3,myVector.end());  
for (int num : myVector)  
    cout << num << " ";
```

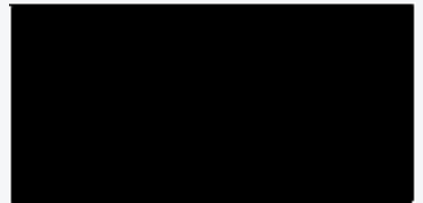
1 3 4



تابع clear()

این تابع کل عناصر بردار را حذف می‌کند.

```
vector<int> myVector = {10, 20, 30, 40, 50};  
myVector.clear();  
for (int num : myVector)  
    cout << num << " ";
```



تابع swap(otherVector)

این تابع محتوای دو بردار را جابجا می‌کند.

```
vector<int> vec1 = {10, 20, 30, 40, 50};  
vector<int> vec2 (3, 1);  
vec1.swap(vec2);
```



❖ این کتابخانه شامل توابع مفیدی برای کار بر روی ساختار داده‌های مختلف از جمله **vector** دارد. برای استفاده از این توابع باید کتابخانه **algorithm** را به برنامه ضمیمه کرد. برخی از این توابع عبارتند از:

❖ تابع **min_element**: پیدا کردن کمترین مقدار در بین داده‌ها

❖ تابع **max_element**: پیدا کردن بیشترین مقدار در بین داده‌ها

❖ تابع **find**: جستجو کردن یک داده خاص در بین داده‌ها

❖ تابع **binary_search**: بررسی وجود یا عدم وجود داده خاص با استفاده از جستجوی دودویی

❖ تابع **sort**: مرتب کردن داده‌ها

مثال



مثال ۶: برنامه روبرو نحوه استفاده از کلاس `vector` و قابلیت‌های این کلاس را نشان می‌دهد.

```
#include <iostream>
#include <vector>
#include <algorithm>
```

```
using namespace std;
```

```
void outputVector( const vector< int > & ); // display the vector
void inputVector( vector< int > & ); // input values into the vector
```

```
int main() {
    vector<int> vec1;
    vector<double> vec2 = {0.5, 1, 1.5, 2, 2.5};
    vector<int> vec3(6);

    cout << "vector 1: ";
    for(int i = 0; i < vec1.size(); i++)
        cout << vec1[i] << " ";
    cout << endl;

    cout << "vector 2: ";
    for (double num:vec2)
        cout << num << " ";
    cout << endl;
```



```

cout << "vector 3: ";
vector<int>::iterator it;
for (it = vec3.begin(); it != vec3.end(); ++it)
    cout << *it << " ";
cout << endl;

for (int i = 0; i < 10; ++i)
    vec1.push_back(i);

cout << "Enter " << vec3.size() << " integers for vector 3:\n";
inputVector(vec3);

cout << "vector 1 after adding elements: ";
outputVector(vec1);
cout << "vector 3 after the user enters elements: ";
outputVector(vec3);

cout << "vector 1: size = " << vec1.size() << ", Capacity = " << vec1.capacity() << endl;
cout << "vector 2: size = " << vec2.size() << ", Capacity = " << vec2.capacity() << endl;
cout << "vector 3: size = " << vec3.size() << ", Capacity = " << vec3.capacity() << endl;

```



```
cout << "First element of vector 1: " << vec1.front() << ", Last element of vector 1: " << vec1.back() << endl;
cout << "First element of vector 2: " << vec2.front() << ", Last element of vector 2: " << vec2.back() << endl;
cout << "First element of vector 3: " << vec3.front() << ", Last element of vector 3: " << vec3.back() << endl;
```

```
cout << "Forth element of vector 3 (vect3[3]): " << vec3[3] << endl;
cout << "Ninth element of vector 3 (vect3[8]): " << vec3[8] << endl;
```

```
try
{
    cout << "Attempt to display ninth element of vector 3 by vec3.at(8)" << endl;
    cout << vec3.at( 8 ) << endl; // ERROR: out of range
}
catch ( out_of_range &ex )
{
    cout << "An exception occurred: " << ex.what() << endl;
}
```

```
vec1.insert(vec1.begin()+7,123);
vec3.insert(vec3.end(),vec1.begin()+2,vec1.begin()+4);
```



```
cout << "vector 1 after elemet insertion: ";  outputVector(vec1);
cout << "vector 3 after element insertion: ";  outputVector(vec3);
```

```
if (!vec1.empty())
    vec1.pop_back();
```

```
if (vec1.size() > 3)
    vec1.erase(vec1.begin() + 2);
```

```
if (vec3.size() > 5)
    vec3.erase(vec3.begin() + 4, vec3.end());
```

```
cout << "vector 1 after removing some elements: ";
outputVector(vec1);
cout << "vector 3 after removing some elements: ";
outputVector(vec3);
```

```
vector<int> vec4;
vec4 = vec1;
cout << "vector 4: ";  outputVector(vec4);
```

```
if ( vec1 == vec4 )
    cout << "vector 1 and vector 4 are equal" << endl;
```


مثال



```
vec1.clear();  
cout << "vector 1 after removing all elements: ";  outputVector(vec1);
```

```
vec3.swap(vec4);  
cout << "vector 3 after swapping: ";  outputVector(vec3);  
cout << "vector 4 after swapping: ";  outputVector(vec4);
```

```
auto minIt = min_element(vec4.begin(),vec4.end());  
cout << "minimum element in vector 4: " << *minIt << endl;  
auto maxIt = max_element(vec4.begin(),vec4.end());  
cout << "maximum element in vector 4: " << *maxIt << endl;
```

```
cout << "Enter a key for searching in vector 4: ";  
int key;  cin >> key;  
it = find(vec4.begin(),vec4.end(),key);  
if (it != vec4.end())  
    cout << key << " is found!\n";  
else  
    cout << key << " is not found!\n";
```

```
sort(vec4.begin(),vec4.end());  
cout << "vector 4 after sorting: ";  outputVector(vec4);  
return 0;
```

```
}
```



// output vector contents

```
void outputVector( const vector< int > &array )
{
    size_t i;
    for ( i = 0; i < array.size(); i++ )
        cout << array[i] << " ";

    cout << endl;
} // end function outputVector
```

// input vector contents

```
void inputVector( vector< int > &array )
{
    for (int &num: array)
        cin >> num;
} // end function inputVector
```

مثال

خروجی برنامه:

```
"C:\Users\Maryam\courses\Computer Basics\vector2\bin\Debug\vector2.exe"
vector 1:
vector 2: 0.5 1 1.5 2 2.5
vector 3: 0 0 0 0 0 0
Enter 6 integers for vector 3:
17 38 12 24 2 40
vector 1 after adding elements: 0 1 2 3 4 5 6 7 8 9
vector 3 after the user enters elements: 17 38 12 24 2 40
vector 1: size = 10, Capacity = 16
vector 2: size = 5, Capacity = 5
vector 3: size = 6, Capacity = 6
First element of vector 1: 0, Last element of vector 1: 9
First element of vector 2: 0.5, Last element of vector 2: 2.5
First element of vector 3: 17, Last element of vector 3: 40
Forth element of vector 3 (vect3[3]): 24
Ninth element of vector 3 (vect3[8]): 0
Attempt to display ninth element of vector 3 by vec3.at(8)
An exception occurred: vector::_M_range_check: __n (which is 8) >= this->size() (which is 6)
vector 1 after elemet insertion: 0 1 2 3 4 5 6 123 7 8 9
vector 3 after element insertion: 17 38 12 24 2 40 2 3
vector 1 after removing some elements: 0 1 3 4 5 6 123 7 8
vector 3 after removing some elements: 17 38 12 24
vector 4: 0 1 3 4 5 6 123 7 8
vector 1 and vector 4 are equal
vector 1 after removing all elements:
vector 3 after swapping: 0 1 3 4 5 6 123 7 8
vector 4 after swapping: 17 38 12 24
minimum element in vector 4: 12
maximum element in vector 4: 38
Enter a key for searching in vector 4: 17
17 is found!
vector 4 after sorting: 12 17 24 38

Process returned 0 (0x0)   execution time : 896.041 s
Press any key to continue.
```



Strings

- ❖ در **C++** دنباله‌ای از کاراکترها رشته نامیده می‌شود. یک رشته ترکیبی از حروف، ارقام و کاراکترهای ویژه مانند +، &، \$ و... است. رشته‌ها برای ذخیره، بازیابی و دستکاری کلمات و متون استفاده می‌شوند.
- ❖ در زبان **C** رشته‌ها به صورت آرایه‌ای از کاراکترها ایجاد می‌شود. چون زبان **C++** همچنان از برنامه‌های نوشته شده به زبان **C** پشتیبانی می‌کند، از رشته‌هایی با این سبک هم همچنان می‌توان در **C++** استفاده کرد که معمولاً اصطلاحاً **C-style String** برای آنها به کار می‌رود.
 - ✓ بسیاری از نکات ذکر شده در مورد آرایه‌ها در مورد رشته‌ها هم صدق می‌کند.
 - ✓ برای مشخص کردن انتهای رشته در این پیاده‌سازی از کاراکتر خاصی به نام **NULL** که با **'\0'** نشان داده می‌شود، استفاده می‌شود. (چون معمولاً طول کلماتی (متون) که در متغیرهای رشته‌ای نگهداری می‌شوند، متفاوت است).
 - ✓ برای رشته‌ها همیشه باید به اندازه یک کاراکتر بیشتر فضا گرفت تا فضا برای نگهداری کاراکتر انتهای رشته (**'\0'**) وجود داشته باشد.
- ❖ در **C++** یک کلاس **string** در کتابخانه استاندارد آن ارائه شده است که امروزه برای پردازش رشته‌ها بیشتر در برنامه‌ها مورد استفاده قرار می‌گیرد.
 - ✓ کلاس **string** در **C++** امکانات متنوعی را در اختیار برنامه‌نویسان قرار می‌دهد و جلوی بسیاری از خطاها که در **C-style string** وجود دارد را می‌گیرد.

مقداردهی اولیه رشته‌های به سبک C

C-style string initialization



```
char s1[10] = "computer";
```

S[0]	S[2]	S[4]	S[6]	S[8]					
c	o	m	p	u	t	e	r	\0	?
	S[1]	S[3]	S[5]	S[7]	S[9]				

```
char s[ ] = "computer";
```

S[0]	S[2]	S[4]	S[6]	S[8]				
c	o	m	p	u	t	e	r	\0
	S[1]	S[3]	S[5]	S[7]				

```
Char s[ ] = {'c', 'o', 'm', 'p', 'u', 't', 'e', 'r', '\0'};
```

S[0]	S[2]	S[4]	S[6]	S[8]				
c	o	m	p	u	t	e	r	\0
	S[1]	S[3]	S[5]	S[7]				



❖ برای دریافت و نمایش رشته‌های به سبک C می‌توان از دستورات **cin** و **cout** استفاده کرد:

```
char name[11];
cin >> name;
cout << name;
```

✓ دقت کنید اگر تعداد کاراکترهای ورودی بیش از ۱۰ (یکی کمتر از طول رشته) کاراکتر باشد، رفتار برنامه قابل پیش‌بینی نیست.

✓ دقت کنید که **cin** رسیدن به اولین **space** یا **tab** یا **newline** را خاتمه رشته تلقی می‌کند و کاراکتر **'\0'** را به انتهای رشته اضافه می‌کند.

❖ برای دریافت رشته از دستور **cin.get** و **cin.getline** نیز می‌توان استفاده کرد:

```
cin.get(['کاراکتر خاتمه ورود رشته', حداکثر طول رشته, نام متغیر رشته‌ای])
cin.getline(['کاراکتر خاتمه ورود رشته', حداکثر طول رشته, نام متغیر رشته‌ای])
```

✓ دقت کنید دستور **cin.get** کاراکتر خاتمه رشته را از بافر حذف نمی‌کند ولی **cin.getline** کاراکتر خاتمه رشته را از بافر حذف می‌کند.

✓ دقت کنید اگر تعداد کاراکترهای دریافتی بیش از یکی کمتر از طول رشته باشد، رفتار برنامه قابل پیش‌بینی نیست.

❖ گذاشتن علامت خاتمه ورود رشته اختیاری است و در صورت مشخص نکردن آن به طور پیش‌فرض، **'\n'** نشان دهنده پایان ورود رشته است.

```
char name[21];
cin.get(name, 20);
cin.get(name, 20, '.');
```

✓ در مثال فوق دستور اول **cin.get** حداکثر ۲۰ کاراکتر را از ورودی دریافت می‌کند و در متغیر **name** قرار می‌دهد. انتهای ورود اطلاعات با **Enter** مشخص می‌شود.

✓ در مثال فوق دستور دوم **cin.get** حداکثر ۲۰ کاراکتر را از ورودی دریافت می‌کند و در متغیر **name** قرار می‌دهد. انتهای ورود اطلاعات با **'.'** مشخص می‌شود.

❖ تفاوت اصلی دستور **cin** و **cin.get** (**cin.getline**) این است که با **cin.get** (**cin.getline**) رشته دریافتی می‌تواند حاوی **space**، **tab** و **newline** باشد.



❖ کتابخانه **cstring** (**string.h**) توابعی برای کار با رشته‌ها (**C-style string**) در دسترس قرار می‌دهد که کار با رشته‌ها را راحت‌تر می‌کند. برای استفاده از این توابع باید این فایل را به برنامه ضمیمه کرد. برای این کار باید دستورالعمل زیر را قبل از شروع تابع **main** قرار داد.

```
#include <iostream>
```

```
#include <cstring>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
...
```

```
return 0;
```

```
}
```



تابع strcpy(s1,s2)

این تابع دو رشته را به عنوان ورودی پذیرفته و رشته دوم را در رشته اول کپی می‌نماید و نهایتاً مقدار رشته اول را برمی‌گرداند.

```
char s1[20];  
char s2[20]="Hello world";  
//s1 = s2; ERROR  
strcpy(s1,s2);  
cout << s1 << endl;
```

Hello world

تابع strncpy(s1,s2,n)

این تابع دو رشته و یک عدد صحیح مثبت (n) را به عنوان ورودی پذیرفته و حداکثر n کاراکتر از رشته دوم را در رشته اول کپی می‌نماید و نهایتاً مقدار رشته اول را برمی‌گرداند.

```
char s1[6];  
char s2[20]="Hello world";  
Strncpy(s1,s2,5);  
cout << s1 << endl;
```

Hello



تابع `strcat(s1,s2)`

این تابع دو رشته را به عنوان ورودی پذیرفته و رشته دوم را به انتهای رشته اول اضافه می‌نماید و نهایتاً مقدار رشته اول را برمی‌گرداند.

```
char s1[30]="C++ programming";  
char s2[10]=" is fun!";  
strcat(s1,s2);  
cout << s1 << endl;
```

C++ programming is fun!

تابع `strncat(s1,s2,n)`

این تابع دو رشته و یک عدد صحیح مثبت (n) را به عنوان ورودی پذیرفته و حداکثر n کاراکتر از رشته دوم را به انتهای رشته اول اضافه می‌نماید و نهایتاً مقدار رشته اول را برمی‌گرداند.

```
char s1[30]="C++ programming";  
char s2[10]=" is fun!";  
strncat(s1,s2,3);  
cout << s1 << endl;
```

C++ programming is



تابع strcmp(s1,s2)

این تابع دو رشته را به عنوان ورودی پذیرفته و رشته‌ها را با هم مقایسه نموده و اگر دو رشته برابر باشد، مقدار صفر و اگر رشته اول از رشته دوم کوچکتر باشد، عددی منفی و اگر بزرگتر باشد، عدد مثبت برمی‌گرداند.

```
char s1[20]="C++ programming";
char s2[20]="C++ programing";
//s1 == s2 ERROR
cout << strcmp(s1,s2) << endl;
Cout << strcmp(s2,s1) << endl;
```

1
-1

تابع strncmp(s1,s2,n)

این تابع دو رشته و یک عدد صحیح مثبت (n) را به عنوان ورودی پذیرفته و حداکثر n کاراکتر از رشته اول را با n کاراکتر از رشته دوم مقایسه نموده و اگر دو رشته برابر باشد، مقدار صفر و اگر کوچکتر باشد، عددی منفی و اگر بزرگتر باشد، عدد مثبت برمی‌گرداند.

```
char s1[20]="C++ programming";
char s2[20]="C++ programing";
cout << strncmp(s1,s2,5) << endl;
```

0



تابع strcmpi(s1,s2)

این تابع دو رشته را به عنوان ورودی پذیرفته و رشته‌ها را بدون در نظر گرفتن کوچکی یا بزرگی حروف با هم مقایسه نموده و اگر دو رشته برابر باشد، مقدار صفر و اگر رشته اول از رشته دوم کوچکتر باشد، عددی منفی و اگر بزرگتر باشد، عدد مثبت برمی‌گرداند.

```
char s1[20]="C++ Programming";  
char s2[20]="C++ PROGRAMMING";  
//s1 == s2 ERROR  
cout << strcmpi(s1,s2) << endl;
```

0

تابع strlen(s1)

این تابع یک رشته را به عنوان ورودی پذیرفته و طول رشته را برمی‌گرداند.

```
char s1[20]="C++ Programming";  
cout << strlen(s1) << endl;
```

15

مثال



مثال ۷: برنامه‌ای بنویسید که نام و نمرات دانشجویان یک کلاس ۴۰ نفره را دریافت کند و نام دانشجویی که بیشترین نمره را گرفته است چاپ کند.

```
#include <iostream>
#include <cstring>
using namespace std;
int main()
{
    const int n = 40;
    char Names[n][31], MaxName[31];
    double Grades[n], max = 0;
    for( int i = 0; i < n; ++i)
    {
        cin.getline(Names[i], 30);
        cin >> Grades[i];
        cin.ignore(1000, '\n'); // Discards the remaining characters in the input buffer
        cin.clear(); // Reset the stream error flags
        if (Grades[i] > max)
        {
            max = Grades[i];
            strcpy(MaxName, Names[i]);
        }
    }
    cout << MaxName;
    return 0;
}
```

string class



❖ کتابخانه استاندارد **string** امکان استفاده از کلاس **string** و همچنین توابع کار با این رشته‌ها را در دسترس قرار می‌دهد که کار با رشته‌ها را راحت‌تر کرده است. برای استفاده از این توابع باید این فایل را به برنامه ضمیمه کرد. برنامه زیر یک مثال ساده از کاربرد این کلاس در برنامه را نشان می‌دهد.

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
    string s = "hello";
    string s2 ;
    cout << s << endl;
    cin >> s2;
    cout << s2 << endl;
    cin.ignore(1000, '\n');
    cin.clear();
    getline(cin, s2);
    cout << s2 << endl;
    s = s2;
    cout << s << endl;
    return 0;
}
```

```
hello
Welcome to C++!
Welcome
Welcome to C++!
Welcome to C++!
Welcome to C++!
```



تابع assign

این تابع یک رشته را به عنوان ورودی پذیرفته و آن رشته را در رشته اول کپی می‌نماید.

```
string s1;
string s2="Hello world";
s1 = s2; // S1.assign(s2);
cout << s1 << endl;
```

Hello world

دسترسی به کاراکترها با []

مانند آرایه‌ها با عملگر [] می‌توان به کاراکترهای یک رشته دسترسی پیدا کرد.

```
string s1="hello world";
s1[0] = 'H';
cout << s1 << endl;
cout << s1[4];
```

Hello world

o

string class



تابع append

این تابع یک رشته را به عنوان ورودی پذیرفته و رشته دوم را به انتهای رشته اول اضافه می نماید.

```
string s1 = "C++ programming" ;  
string s2 = " is fun!";  
s1.append(s2);  
//s1 = s1 + s2; or  s1 += s2;  
cout << s1 << endl;
```

C++ programming is fun!

تابع length

این تابع طول رشته را برمی گرداند.

```
string s1 = "C++ programming";  
cout << s1.length();
```

15

string class



تابع compare

این تابع یک رشته را به عنوان ورودی پذیرفته و رشته‌ها را با هم مقایسه نموده و اگر دو رشته برابر باشد، مقدار صفر و اگر رشته اول از رشته دوم به لحاظ الفبایی کوچکتر باشد، عددی منفی و اگر بزرگتر باشد، عدد مثبت برمی‌گرداند. می‌توان برای مقایسه از عملگرهای رابطه‌ای $< > =$ استفاده کرد.

```
string s1 = "C++ programming" ;
string s2 = "C++ programming";
cout << S1.compare(s2);
s2 = "string";
cout << S1.compare(s2);
```

0
-1

تابع substr

این تابع دو عدد صحیح مثبت را به عنوان ورودی پذیرفته و از موقعیت عدد اول به اندازه عدد دوم از رشته جدا می‌کند و برمی‌گرداند.

```
string s1 = "C++ programming" ;
string s2;
s2 = s1.substr(4,7);
cout << s2 << endl;
```

program

تمرین



1. برنامه‌ای بنویسید که رشته s و کاراکتر ch را از ورودی دریافت کند و بعد بررسی کند که آیا کاراکتر در رشته وجود دارد یا نه؟ در صورت وجود داشتن مکان کاراکتر و در صورت نبودن «not found» را چاپ کند.
2. برنامه‌ای بنویسید که رشته‌ای از ورودی دریافت کند و سپس تعداد کلمات رشته را شمارش کند. (بین کلمات فضای خالی، ویرگول و نقطه قرار دارد).
3. برنامه‌ای بنویسید که رشته‌ای را از ورودی دریافت کند و سپس حروف کوچک را به حروف بزرگ تبدیل کند.
4. برنامه‌ای بنویسید که رشته‌ای از ورودی دریافت کند و سپس تعداد کلمات ۴ حرفی رشته را شمارش کند. (بین کلمات فضای خالی، ویرگول و نقطه قرار دارد).
5. برنامه‌ای بنویسید که رشته s1 و رشته s2 را از ورودی دریافت کند و بعد بررسی کند که آیا رشته s2 در رشته s1 وجود دارد یا نه؟ در صورت وجود داشتن مکان رشته و در صورت نبودن «not found» را چاپ کند.
6. برنامه‌ای بنویسید که ۱۰۰ نام از ورودی دریافت کند و سپس نامها را به ترتیب حروف الفبا چاپ کند.

با سپاس از توجه شما

